

PilotFish Conditional Node Router – Complete Reference (XCS / 26R1-compatible)

Derived from PilotFish XCS source (not in 26R1.11 WAR decompile)

August 4, 2026

At a glance (plain English)

In **Route v2**, this routing node is a **switchboard with named ports**. Traffic arrives on a stable input port; each output port has a condition (OGNL expression, XPath, or a catch-all **Otherwise**). You choose whether **all** matching branches fire, only the **first** match, or matches rotate in **round-robin** order.

Conditional Node Router

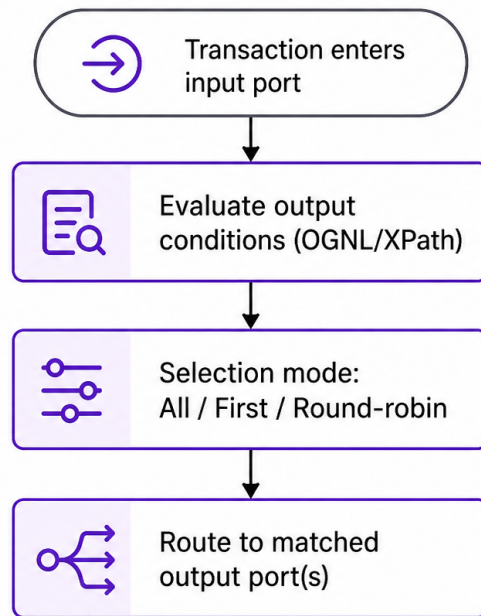


Figure 1: At a glance (plain English)

- **Defines** named input/output ports (v2 graph wiring).
- **Evaluates** OGNL or XPath conditions per output port.
- **Selects** outputs via All Matches, First Match, or Round Robin.

- **Caches** XML temporarily when any XPath condition is present.

Provenance: XCS tree source — not present in `eip.war.hs.26R1.11` decompile. Documented as **XCS / 26R1-compatible** routing module.

Purpose

This document is a **deep dive** into the **Conditional Node Router** (`ConditionalNodeRoutingModule`): port model, condition evaluation, selection modes, XPath caching, and configuration — from XCS source because the class is **not** in the 26R1.11 WAR bytecode extract.

Provenance	Value
WAR	Not in 26R1.11 decompile
Source	PilotFish Sandbox/PilotFish_V2/pilotfishdevelopment-
Ports config item	com.pilotfish.utils.ccp.ConditionalNodeRou
Module type	ROUTING (ModuleType.ROUTING)
Type string	Conditional Node Router
Description	Routes v2 transactions through variable named inputs and conditionally selected outputs.

Implements `NodeRoutingModule` with stable port IDs for v2 route graphs.

1. Conceptual model

Unlike legacy stage-level routing, v2 **nodes** expose explicit **ports**. This module evaluates each output port's condition against the current `TransactionData` (and cached XML for XPath), then returns a list of **output port IDs** for the engine to activate.

```
Transaction arrives (input port)
  |
  v
[Any XPath output?] --> cache body to default Cache (rewind stream)
  |
  v
For each output port (except Otherwise first pass):
  OGNL / Enhanced expr OR Saxon XPath boolean
  |
  v
Collect matching port IDs
  |
  v
If none matched -> Otherwise port (if defined)
  |
  v
Apply SelectionMode (ALL / FIRST / ROUND_ROBIN)
  |
  v
```

Return List<String> port IDs
|
v
Restore original data stream from cache (if used)

2. High-level behavior (`selectOutputPorts`)

Public entry `selectOutputPorts(TransactionData data)`:

1. `prepareXPathData(data)` — if any output uses XPath, copy stream to cache and replace data stream with cache input (section 4).
2. Delegate to `selectOutputPorts(data, xpathData)`.
3. **Finally:** if cache used, reset `data.setDataStream(xpathData.getInputStream())`.

Core selection algorithm:

1. Initialize `matches` list; track first `OTHERWISE` output.
 2. Iterate **output ports** in definition order:
 - Skip evaluation for `OTHERWISE` during loop (stored for fallback).
 - If `matches(output, data, xpathData) → add output.getId()`.
 - **FIRST_MATCH** → break after first add.
 3. If `matches` empty and `otherwise != null` → add `otherwise ID`.
 4. **ROUND_ROBIN** with `matches.size() > 1` → keep single ID at `index = floorMod(sequence++, size)`.
 5. Return `matches` (port ID strings).
-

3. Condition evaluation

3.1 OGNL / enhanced expressions

Method `matchesOgnl(condition, data)`:

- If `EnhancedExpressionUtils.isEnhancedValue(condition) → ConfigurationManager.getEnhancedVal`
`data, route context`).
- Else → `OGNLExpressionHandler.getObject(condition, data, route context, empty`
`map)`.
- **True if:** value is `Boolean.TRUE`, or non-null string `"true"` (case insensitive).
- Runtime errors → `EIPEException` with condition text.

Same enhanced-expression semantics as ordinary v2 connections (per class Javadoc).

3.2 XPath

Method `matchesXPath(output, xpathData)`:

- Requires non-null cache; else error.
- `SaxonXPathEvaluator.createForDocument(cache input stream)`.
- Register output port namespace prefix/URI pairs.

- Evaluate `boolean( + condition + )` via `evaluateToString` → `Boolean.parseBoolean`.

3.3 Otherwise

Ports with `NodeRoutingConditionType.OTHERWISE` are not evaluated in the main loop; used only when no other port matched. Configuration completeness allows **at most one** Otherwise port.

4. XPath caching (`prepareXPathData`)

When any output port has condition type XPath:

1. Require non-null `data.getDataStream()`.
2. `EIPCacheManager.getInstance().getDefaultCache(data)`.
3. `DataUtil.copyAndClose(stream, cache output)`.
4. `data.setDataStream(cache.getInputStream())`.
5. Return cache handle for evaluators.

If no XPath ports, returns `null` (no copy). Stream restored in outer **finally**.

5. Configuration reference

5.1 Module options

UI label	Tag	Type	Default	Required	Nuances
Selection	<code>SelectionMode</code>	Choice	<code>ALL_MATCHES</code>	No	<code>ALL_MATCHES</code> , <code>FIRST_MATCH</code> , <code>ROUND_ROBIN</code>
Ports	<code>RoutingPorts</code>	Conditional	<code>DefaultRoutingPorts</code> Input/Output	Yes	Single input allowed (<code>allowMultipleInputs=false</code>)

Selection mode semantics

Mode	Behavior
<code>ALL_MATCHES</code>	Every port whose condition is true is selected
<code>FIRST_MATCH</code>	Stop after first true condition in port order
<code>ROUND_ROBIN</code>	When multiple matches, emit one ID rotating via <code>AtomicInteger</code>

Invalid/missing selection string → defaults to `ALL_MATCHES`.

5.2 Ports configuration (RoutingPorts)

Serialized via `ConditionalNodeRoutingPortsConfigurationItem`:

- **Inputs:** named input ports (`NodeRoutingInput`); default single port "Input".
- **Outputs:** named outputs with:
 - **OGNL** — condition string in `<condition>` element, `type` attribute.
 - **XPATH** — condition + optional `<namespaces>` prefix/uri pairs.
 - **OTHERWISE** — catch-all (`fallback="true"` legacy XML supported on read).

Default value: one input "Input", one output "Output" with condition "true".

Completeness rules (`isRoutingConfigurationComplete`):

- Non-otherwise outputs require non-empty condition and type.
- XPath outputs require valid namespace prefix/URI pairs.
- At most one Otherwise port.

5.3 Inherited AbstractEIPModule

Routing modules use standard configuration manager initialization; no `ExecuteProcessor`-style conditional flag in this class.

6. API surface (runtime)

Method	Role
<code>getInputPorts()</code> / <code>getOutputPorts()</code>	Delegate to ports configuration item
<code>setPorts(inputs, outputs)</code>	Programmatic port mutation
<code>getSelectionMode()</code> / <code>setSelectionMode()</code>	Access selection enum
<code>selectOutputPorts(TransactionData)</code>	Core routing decision

`roundRobinSequence` is `transient`; resets on module deserialization until first use (lazy re-init to 0).

7. Worked example — first match with otherwise

Ports:

Output	Type	Condition
HighPriority	OGNL	<code>@java.lang.Integer@parseInt(TrAttr['priority']) > 5</code>
Standard	OGNL	<code>true</code>
Fallback	OTHERWISE	—

Selection: **FIRST_MATCH**

- Transaction with `priority=10` → selects `HighPriority` only.
 - Transaction with `priority=1` → Standard matches first (`true`) before Fallback would be needed.
 - Transaction where all false → Fallback Otherwise port.
-

8. Known quirks and caveats

1. **Not in 26R1.11 WAR** — available in XCS / v2 route tooling; verify deployment matches source version.
 2. **Round robin only when multiple matches** — single match bypasses rotation.
 3. **Port order matters** for `FIRST_MATCH`.
 4. **XPath boolean wrapper** — engine wraps condition as `boolean(...)`; write inner expression accordingly.
 5. **Stream side effect** — XPath path temporarily replaces data stream; restored after selection unless I/O fails.
 6. **OGNL true strings** — non-boolean truthy values except `"true"` string do not match.
-

9. Operational recommendations

Goal	Guidance
v2 graphs	Use stable port IDs when refactoring routes
XML routing	Prefer XPath ports only when body is XML; triggers cache copy
Default path	Always configure exactly one Otherwise for exhaustiveness
Load spreading	Use <code>ROUND_ROBIN</code> among peer outputs with identical conditions
Testing	Unit-test OGNL with representative <code>TransactionData</code> attributes

10. Source index (XCS)

Topic	Path
Conditional Node Router	<code>.../ConditionalNodeRoutingModule.java</code>
Ports configuration item	<code>.../ConditionalNodeRoutingPortsConfigurationItem.java</code>
Console UI component	<code>.../ConditionalNodeRoutingPortsConfigurationComponent.java</code>

Base path: `PilotFish Sandbox/PilotFish_V2/pilotfishdevelopment-xcs-b313c66ccd5f/`

11. Pipeline provenance

PilotFish XCS source (NOT eip.war.hs.26R1.11 decompile)

-> ConditionalNodeRoutingModule.java

-> Documents/Routing/26R1.11/PilotFish-Conditional-Node-Router-Reference-26R1.11.(md|pdf)

Deep-dive documentation from XCS source – Conditional Node Router – 26R1-compatible – August 2026.